COMP20003 OPEN-BOOK EXAM CHEAT SHEET & REFERENCE MANUAL

Part I: Visual Topic Tree Index | Part II: Formulas, Lookup Tables & Code Templates

IN-SCOPE (Core Exam Syllabus)	OUT-OF-SCOPE (Never Tested)
• C Pointers, malloc/free, Linked Lists, Dynamic Arrays • Loop Summations, Recurrences, Unrolling, Master Theorem • BST, AVL Rotations, Trie, Patricia Tree, Huffman Tree • Priority Queue, Binary Heap (1-based), Bottom-up Heapify • Hash Tables: Separate Chaining, Linear Probing, Double Hashing • Sorts: Selection, Insertion, Quicksort (Hoare), Mergesort, Heapsort • Graphs: BFS/DFS, UCS, Dijkstra, Bellman-Ford, A*, Floyd-Warshall • MST: Prim, Kruskal, Disjoint Sets (Union-Find), P vs NP	• C Library API Parameter Syntax (qsort, memcpy, bsearch) • Red-Black Tree Recoloring, Splay Tree, B-Tree Deletion • Cryptographic Hash (Preimage / Collision Resistance) • RSA Encryption, Euler's Totient Phi(N), Extended Euclid • Shannon-Fano Coding, Max-Flow Min-Cut, String KMP • Writing multi-page full C implementations from scratch

PART I · VISUAL TOPIC TREE INDEX

```
[!!!] = Core / Must-know    [!! ] = Important / Frequent    [!  ] = Basic / Edge case

[0.0] COMP20003 ALGORITHMS AND DATA STRUCTURES
│
├── [1.0] C Memory & Linear Data Structures
│   ├── [1.1] Pointers & Addressing
│   │   ├── [1.1.1] Pointer dereferencing (*p) & address-of (&x)        [!!!]
│   │   ├── [1.1.2] Pointer arithmetic & traversal (ptr++)              [!!!]
│   │   ├── [1.1.3] Multi-value return via pointer parameters           [!!!]
│   │   ├── [1.1.4] Double pointers (T **) head modification            [!! ]
│   │   └── [1.1.5] Generic (void *) & comparator function pointers     [!  ]
│   │
│   ├── [1.2] Dynamic Memory Management
│   │   ├── [1.2.1] Heap allocation (malloc, realloc, sizeof)           [!!!]
│   │   ├── [1.2.2] Null-terminated string allocation (len + 1)         [!!!]
│   │   ├── [1.2.3] Base-address invariant for free(ptr)                [!!!]
│   │   └── [1.2.4] Memory leaks, dangling pointers & buffer overflow   [!!!]
│   │
│   └── [1.3] Linear Structures
│       ├── [1.3.1] Singly & doubly linked list insertion & deletion    [!!!]
│       ├── [1.3.2] Dynamic array geometric doubling reallocation       [!! ]
│       └── [1.3.3] Stack (LIFO) & Queue (FIFO) ADT operations          [!! ]
│
├── [2.0] Asymptotic Complexity & Recurrences
│   ├── [2.1] Asymptotic Notation
│   │   ├── [2.1.1] Formal Big-O, Big-Omega & Big-Theta definitions     [!!!]
│   │   ├── [2.1.2] Asymptotic growth rate ordering hierarchy           [!! ]
│   │   └── [2.1.3] Best, worst, average time & extra space bounds      [!!!]
│   │
│   ├── [2.2] Loop Summation Analysis
│   │   ├── [2.2.1] Arithmetic series nested loop summation             [!!!]
│   │   └── [2.2.2] Geometric series nested loop summation (i *= 2)     [!!!]
│   │
│   └── [2.3] Recurrence Relations
│       ├── [2.3.1] Recurrence formulation from recursive code          [!!!]
│       ├── [2.3.2] Unrolling / repeated substitution resolution       [!!!]
│       └── [2.3.3] Master Theorem classification & application        [!! ]
│
├── [3.0] Trees & Balanced Search Trees
│   ├── [3.1] Binary Tree Fundamentals
│   │   ├── [3.1.1] Tree height, depth & complete tree bounds           [!! ]
│   │   └── [3.1.2] Pre-order, In-order, Post-order & Level-order trace [!!!]
│   │
│   ├── [3.2] Binary Search Trees (BST)
│   │   ├── [3.2.1] BST ordering invariant & sequential insertion trace [!!!]
│   │   ├── [3.2.2] BST search & 2-child node deletion (in-order swap)  [!!!]
│   │   └── [3.2.3] Balanced O(log n) vs degenerate chain O(n) analysis [!!!]
│   │
│   ├── [3.3] AVL Balanced Trees
│   │   ├── [3.3.1] Node balance factor computation: H(left) - H(right) [!!!]
│   │   ├── [3.3.2] Imbalance case classification (LL, RR, LR, RL)      [!!!]
│   │   └── [3.3.3] Single (L, R) & double (LR, RL) tree rotations      [!!!]
│   │
│   └── [3.4] Prefix & Coding Trees
│       ├── [3.4.1] Multi-way Trie character/bit lookup & insertion     [!! ]
```

```
|       ├── [3.4.2] Patricia / Radix tree prefix splitting & bit match   [!! ]
|       └── [3.4.3] Huffman frequency tree & prefix code generation      [!  ]
|
├── [4.0] Priority Queues & Binary Heaps
|   ├── [4.1] Priority Queue ADT
|   |   ├── [4.1.1] Enqueue & delete-min / delete-max operations         [!! ]
|   |   ├── [4.1.2] Array/List vs Heap PQ complexity comparison          [!!!]
|   |   └── [4.1.3] Bounded-key chained hash table Priority Queue        [!!!]
|   |
|   ├── [4.2] Binary Heap Mechanics
|   |   ├── [4.2.1] Heap order invariant & complete binary tree shape    [!!!]
|   |   ├── [4.2.2] 1-based array index arithmetic (i/2, 2i, 2i+1)       [!!!]
|   |   ├── [4.2.3] Array vs Linked-List heap representation trade-off   [!!!]
|   |   └── [4.2.4] Insertion upheap & root-deletion downheap trace      [!!!]
|   |
|   └── [4.3] Heap Construction & Selection
|       ├── [4.3.1] Top-down sequential insertion build O(n log n)       [!! ]
|       ├── [4.3.2] Bottom-up heapify (n/2 downheaps) in Theta(n)        [!!!]
|       └── [4.3.3] Heap-based m-th smallest element selection           [!! ]
|
├── [5.0] Hash Tables & Collision Resolution
|   ├── [5.1] Hashing Fundamentals
|   |   ├── [5.1.1] Modulo hashing h(k) = k mod M & prime table size     [!! ]
|   |   └── [5.1.2] Load factor (alpha = n / M) & table rehashing        [!! ]
|   |
|   └── [5.2] Collision Resolution Strategies
|       ├── [5.2.1] Separate Chaining insertion trace & probe count      [!!!]
|       ├── [5.2.2] Linear Probing trace, probe count & clustering       [!!!]
|       ├── [5.2.3] Double Hashing trace & coprime non-zero h2(k) rules  [!!!]
|       └── [5.2.4] Quadratic Probing offset calculation                 [!  ]
|
├── [6.0] Sorting Algorithms
|   ├── [6.1] Sorting Properties & Bounds
|   |   ├── [6.1.1] Stability, In-place & Adaptivity classification      [!!!]
|   |   └── [6.1.2] Best/avg/worst time & extra space master matrix      [!!!]
|   |
|   ├── [6.2] Elementary Iterative Sorts
|   |   ├── [6.2.1] Selection Sort trace, swap count & Theta(n^2) bound  [!! ]
|   |   └── [6.2.2] Insertion Sort trace & adaptive Theta(n) best case   [!! ]
|   |
|   ├── [6.3] Quicksort
|   |   ├── [6.3.1] Hoare two-pointer partition trace & pivot swap       [!!!]
|   |   ├── [6.3.2] Balanced O(n log n) vs sorted O(n^2) recursion tree  [!!!]
|   |   ├── [6.3.3] Randomized & Median-of-Three pivot selection         [!!!]
|   |   ├── [6.3.4] Small-subarray Insertion Sort cutoff optimization    [!! ]
|   |   └── [6.3.5] Quickselect partition-based m-th smallest search     [!! ]
|   |
|   ├── [6.4] Mergesort
|   |   ├── [6.4.1] Two-pointer Merge trace & leftover partition copy    [!!!]
|   |   ├── [6.4.2] Array O(n) vs Linked-List O(1) auxiliary space       [!!!]
|   |   └── [6.4.3] Top-down, Bottom-up & Natural (Adaptive) Mergesort   [!! ]
|   |
|   └── [6.5] Heapsort & Non-Comparison Sort
|       ├── [6.5.1] Heapsort two-phase trace (heapify + root extraction) [!!!]
|       ├── [6.5.2] Heapsort O(1) space, Theta(n log n) & instability    [!!!]
|       └── [6.5.3] Distribution Counting sort O(n + Range)              [!  ]
|
├── [7.0] Graphs & Graph Algorithms
|   ├── [7.1] Graph Representations & Modeling
|   |   ├── [7.1.1] Graph property identification (directed, cyclic, etc)[!! ]
|   |   ├── [7.1.2] Explicit vs Implicit state-space graph generation    [!! ]
|   |   └── [7.1.3] Adjacency Matrix vs Adjacency List space/time bounds [!!!]
|   |
|   ├── [7.2] Unweighted Traversals & DAGs
|   |   ├── [7.2.1] Depth-First Search (DFS) stack/component trace       [!!!]
|   |   ├── [7.2.2] Breadth-First Search (BFS) queue/shortest-hop trace  [!!!]
|   |   └── [7.2.3] DFS-based Topological Sort on DAGs                   [!! ]
|   |
|   ├── [7.3] Shortest Path Algorithms
|   |   ├── [7.3.1] Uniform-Cost Search (UCS) single-target exploration  [!! ]
|   |   ├── [7.3.2] Dijkstra relaxation trace & prev[] path recovery     [!!!]
|   |   ├── [7.3.3] Negative edge failure & positive-offset fallacy      [!!!]
|   |   ├── [7.3.4] Bellman-Ford & negative-weight cycle breakdown       [!!!]
|   |   ├── [7.3.5] A* search pruning & admissible heuristic criteria    [!! ]
```

```
│   │   ├── [7.3.6] Floyd-Warshall 3D matrix & Next[][] path trace       [!!!]
│   │   ├── [7.3.7] Floyd-Warshall negative diagonal cycle detection     [!!!]
│   │   └── [7.3.8] Sparse (V * Dijkstra) vs Dense (Floyd-Warshall) APSP [!!!]
│   │
│   └── [7.4] Minimum Spanning Trees (MST)
│       ├── [7.4.1] MST structural invariants & Minimum Fatness Path     [!!!]
│       ├── [7.4.2] Prim vertex-greedy trace & Dijkstra priority diff    [!!!]
│       ├── [7.4.3] Multi-component Prim adaptation (Spanning Forest)    [!! ]
│       ├── [7.4.4] Kruskal edge-sorted greedy trace & O(E log E) bound  [!!!]
│       └── [7.4.5] Union-Find (Path Compression & Union-by-Rank)        [!!!]
│
└── [8.0] Algorithm Families & Complexity Theory
    ├── [8.1] Algorithmic Design Paradigms
    │   └── [8.1.1] Brute-Force, Divide & Conquer, Greedy & DP selection [!! ]
    │
    └── [8.2] Determinism & NP Complexity Classes
        ├── [8.2.1] Deterministic vs Non-deterministic state transitions [!! ]
        ├── [8.2.2] Decision problems, Class P & Class NP verification   [!! ]
        └── [8.2.3] NP-Hard, NP-Complete reductions & classic problems   [!  ]
```

PART II · OPEN-BOOK REFERENCE TABLES, FORMULAS & TEMPLATES

[1.0] C Memory & Linear Data Structures

Index & Topic	Syntax / Formula / Rule	Critical Exam Trap / Edge Case
[1.1.1] Dereference & Address	*p (value at address p) | &x (address of x)	Uninitialized pointer *p causes Segmentation Fault
[1.1.2] Pointer Arithmetic	ptr + k == base_addr + k * sizeof(*ptr)	After loop for(i=0; i<n; i++) ptr++, ptr points to arr + n (1 past end)
[1.1.3] Multi-Value Return	void fn(int *arr, int n, int *min, int *range)	Write outputs via *min = ...; caller passes &min_val
[1.1.4] Double Pointer	void push(node_t **head, int v) { *head = ... }	Required when modifying caller's head/root pointer without return
[1.1.5] Generic Comparator	int cmp(const void *a, const void *b)	Cast first: const item_t *x = a; return x->key - y->key;
[1.2.1] Heap Allocation	T *arr = malloc(n * sizeof(T));	Always check assert(arr != NULL); sizeof(*arr) is safest
[1.2.2] String Allocation	char *s = malloc((len + 1) * sizeof(char));	MUST allocate +1 byte and set s[len] = '\0'
[1.2.3] free(ptr) Rule	free(base_ptr); // MUST be exact malloc address	Calling free(ptr) after ptr++ -> Heap Corruption + Memory Leak
[1.2.4] Memory Bugs	Leak: lost block | Dangling: use after free / local &x	Set ptr = NULL after free(ptr); never return &local_var

```
/* [1.1.3] Multi-Output Pointer Function Template */
void min_and_range(int *arr, int n, int *min, int *range) {
    int max = arr[0]; *min = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] < *min) *min = arr[i];
        if (arr[i] > max)  max  = arr[i];
    }
    *range = max - *min;
}
```

```
/* [1.3.1] Doubly Linked List Node Deletion Template */
if (curr->prev != NULL) curr->prev->next = curr->next;
else                    list->head = curr->next; /* Deleting head
if (curr->next != NULL) curr->next->prev = curr->prev;
else                    list->foot = curr->prev; /* Deleting tail
free(curr);
```

[2.0] Asymptotic Complexity & Recurrences

Index & Concept	Mathematical Definition / Closed-Form Formula	Result / Exam Rule
[2.1.1] Big-O (Upper Bound)	f(n) in O(g(n)) iff there exists c > 0, n0 >= 1 : for all n >= n0, 0 <= f(n) <= c * g(n)	Asymptotic upper bound (<=)
[2.1.1] Big-Omega (Lower Bound)	f(n) in Omega(g(n)) iff there exists c > 0, n0 >= 1 : for all n >= n0, f(n) >= c * g(n) >= 0	Asymptotic lower bound (>=)
[2.1.1] Big-Theta (Tight Bound)	f(n) in Theta(g(n)) iff f(n) in O(g(n)) AND f(n) in Omega(g(n))	Exact asymptotic growth rate (==)
[2.1.2] Growth Hierarchy	O(1) < O(log n) < O(sqrtn) < O(n) < O(n log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)	Log base change is constant: log2n = Theta(log10n)
[2.2.1] Arithmetic Series	sum(i=1 to n) i = 1 + 2 + 3 + ... + n = n(n + 1) / 2	Nested loop (i++; j < i) -> Theta(n^2)
[2.2.2] Geometric Series	sum(k=0 to m) r^k = (r^(m+1) - 1) / (r - 1) [For r=2: 2^(m+1) - 1]	Outer i *= 2, inner j < i: 1+2+4+...+2^m < 2n -> Theta(n)
[2.2.2] Harmonic / Log Sum	sum(i=1 to n) (1/i) = Theta(log n) | sum(i=1 to n) log(i) = log(n!) = Theta(n log n)	Outer i *= 2 (alone) runs floor(log2 n) + 1 times -> Theta(log n)
[2.3.3] Master Theorem T(n) = a T(n/b) + Theta(n^d)	Compare d with log_b(a) (or compare b^d with a): • Case 1: d > log_b(a) [a < b^d] -> T(n) = Theta(n^d) • Case 2: d = log_b(a) [a = b^d] -> T(n) = Theta(n^d log n) • Case 3: d < log_b(a) [a > b^d] -> T(n) = Theta(n^(log_b a))	Examples: • Binary Search: T(n) = T(n/2) + O(1) -> a=1,b=2,d=0 -> Theta(log n) • Mergesort: T(n) = 2T(n/2) + Theta(n) -> a=2,b=2,d=1 -> Theta(n log n) • 3-Split O(1): T(n) = 3T(n/2) + O(1) -> Theta(n^(log2 3))

[3.0] Trees, BST, AVL, Trie, Patricia & Huffman

Index & Structure	Invariants & Core Formulas	Operations / Rotations / Complexity
[3.1.1] Binary Tree Bounds	Max nodes at level k: 2^k | Max nodes in height h: 2^(h+1) - 1	Complete Binary Tree height: h = floor(log2 n)

Index & Structure	Invariants & Core Formulas	Operations / Rotations / Complexity
[3.1.2] Tree Traversals	Pre-order: Root -> L -> R | In-order: L -> Root -> R Post-order: L -> R -> Root | Level-order: Queue (BFS)	In-order on BST yields sorted ascending sequence Post-order required before freeing tree nodes
[3.2.1] BST Invariant	for all node u: max(Left Subtree) < key(u) < min(Right Subtree)	Avg Search/Insert: O(log n) | Worst (sorted input -> chain): O(n)
[3.2.2] BST Deletion	• 0 children: remove leaf • 1 child: bypass node to child • 2 children: replace key with In-order Predecessor (max of Left) OR In-order Successor (min of Right), then delete that node	Preserves BST ordering invariant in O(height) time
[3.3.1] AVL Balance Factor	BF(u) = Height(Left) - Height(Right) [Valid AVL: BF in {-1, 0, +1}]	Worst-case height strictly O(log n) -> Search/Insert/Delete O(log n)
[3.3.2] AVL 4 Rotations (Triggered at deepest node with |BF| = 2)	• LL (BF(u) = +2, BF(L) >= 0) -> Single Right Rotation on u • RR (BF(u) = -2, BF(R) <= 0) -> Single Left Rotation on u • LR (BF(u) = +2, BF(L) = -1) -> Left Rotate(L), then Right Rotate(u) • RL (BF(u) = -2, BF(R) = +1) -> Right Rotate(R), then Left Rotate(u)	3-Node Visual Rule (keys x < y < z): After ANY rotation, median key y becomes new root, x becomes left child, z becomes right child; 4 subtrees T1 < x < T2 < y < T3 < z < T4 attach in left-to-right order
[3.4.1] Multi-way Trie	1 edge per character/bit; keys stored implicitly along path	Lookup/Insert: O(L) where L = key length (independent of n)
[3.4.2] Patricia / Radix	Compresses single-child chains into bit-array Prefix Stems	Mismatch at bit k -> Split stem: parent gets first k bits; old node & new key branch 0 (left) / 1 (right) on bit k+1
[3.4.3] Huffman Tree	Greedy optimal prefix-free code (no code is prefix of another)	Insert all frequencies into Min-Heap; repeat n-1 times: pop 2 min nodes u,v -> push merged parent (freq = u+v). Time: O(n log n)

[4.0] Priority Queues & Binary Heaps

[4.1.2] Underlying PQ Data Structure	enqueue (Insert)	delete-min / delete-max	Notes / Trade-offs
Unsorted Dynamic Array	O(1) amortized	Theta(n)	Fast insertion; linear scan to find min/max
Unsorted Linked List	O(1)	Theta(n)	Insert at head O(1); must scan entire list for min/max
Sorted Array	O(n)	O(1)	Shift elements on insert O(n); min/max at end of array O(1)
Sorted Linked List	O(n)	O(1)	Linear scan to find insertion point; pop head O(1)
Binary Heap (Array-backed)	O(log n)	O(log n)	Optimal general-purpose PQ; O(1) peek-min/max at index 1
[4.1.3] Chained Hash Table (Bounded Keys: 0 <= key <= r)	O(1)	O(r) worst O(1) amortized	VALID ONLY IF: key range is finite/bounded [0, r], table size M = r + 1, and h(key) = key (1 priority per bucket). FAILS if keys are unbounded with h(k) = k mod 101 (mixes priorities in bucket -> Theta(n))

Index & Heap Mechanic	Formula / Procedure	Complexity & Exam Justification
[4.2.1] Heap Invariants	1. Shape: Complete Binary Tree (filled left-to-right) 2. Order: Max-Heap A[Parent] >= A[i] | Min-Heap A[Parent] <= A[i]	Root at index 1 is always global max (Max-Heap) or min (Min-Heap)
[4.2.2] 1-Based Array Math	Root = 1 | Parent(i) = floor(i / 2) | Left(i) = 2i | Right(i) = 2i + 1	Leaves are at indices floor(n/2) + 1 ... n | Last internal node = floor(n/2)
[4.2.3] Why Array > Linked List	Array computes parent/child indices in O(1) arithmetic	Linked list lacks random access -> finding parent/last node takes O(n), ruining O(log n) heap ops
[4.2.4] Insertion (upheap)	Place new key at A[n+1]; swap with Parent(i) while heap order violated	Worst-case: O(log n) swaps/comparisons along root path
[4.2.4] Deletion (downheap)	Save A[1]; move A[n] to A[1]; n--; swap with higher-priority child	Worst-case: O(log n) swaps/comparisons (2 comparisons per level)
[4.3.1] Top-Down Build	Insert n items one-by-one into empty heap using upheap	Worst-case Time: O(n log n)
[4.3.2] Bottom-Up Heapify	Put n items in array; for i = floor(n/2) down to 1: downheap(i)	Tight Worst-Case Time: Theta(n) (performs n/2 downheaps)
[4.3.3] m-th Smallest (m << n)	• Method A (Full Sort): Sort array + index m -> O(n log n) • Method B (Partial Heapsort): Bottom-up Min-Heap Theta(n) + m × delete-min -> O(n + m log n) • Method C (Size-m Max-Heap): Build size-m Max-Heap Theta(m) + scan n-m items (if < root, replace & downheap) -> O(n log m)	When m << n, Methods B & C beat O(n log n)

[5.0] Hash Tables & Collision Resolution

Strategy & Index	Probe / Bucket Formula	Key Properties, Probe Counting & Exam Rules
[5.1] Fundamentals	h(k) = k mod M | Load Factor alpha = n / M	Choose M = Prime to distribute keys evenly when inputs share common factors
[5.2.1] Separate Chaining	Slot h(k) points to a Linked List of keys	• alpha can exceed 1.0 • Avg Search: O(1 + alpha) | Worst Search (all n keys collide in 1 list): O(n)
[5.2.2] Linear Probing	h_i(k) = (h(k) + i) mod M for i = 0, 1, 2, ...	• Requires alpha <= 1 (degrades severely when alpha > 0.7) • Probe Count: Count EVERY checked slot from home slot until match or empty slot • Suffers from Primary Clustering (adjacent occupied slots merge into long blocks)

Strategy & Index	Probe / Bucket Formula	Key Properties, Probe Counting & Exam Rules
[5.2.3] Double Hashing	h_i(k) = (h1(k) + i * h2(k)) mod M	• Eliminates primary clustering; setting h2(k) = 1 reduces to Linear Probing • TWO MANDATORY RULES for h2(k): 1. h2(k) ≠ 0 for all k (otherwise infinite loop on same slot) 2. gcd(h2(k), M) == 1 (h2(k) and table size M must be coprime so all M slots are reachable)
[5.2.4] Quadratic Probing	h_i(k) = (h(k) + i^2) mod M	Avoids primary clustering, but keys with same h(k) follow identical path (Secondary Clustering)

[6.0] Sorting Algorithms Master Comparison & Mechanics

[6.1] Algorithm	Best Time	Average Time	Worst Time	Extra Space	Stable?	In-Place?	Adaptive?
Selection Sort	Theta(n^2)	Theta(n^2)	Theta(n^2)	O(1)	No	Yes	No (always n(n-1)/2 compares)
Insertion Sort	Theta(n)	Theta(n^2)	Theta(n^2)	O(1)	Yes	Yes	Yes (O(n) on nearly-sorted)
Quicksort	Theta(n log n)	Theta(n log n)	Theta(n^2)	O(log n) stack	No	Yes	No (sorted input -> Theta(n^2) if end pivot)
Mergesort (Array)	Theta(n log n)	Theta(n log n)	Theta(n log n)	O(n)	Yes	No	No
Mergesort (Linked List)	Theta(n log n)	Theta(n log n)	Theta(n log n)	O(1) bottom-up	Yes	Yes	Natural Mergesort: Yes (Best Theta(n))
Heapsort	Theta(n log n)	Theta(n log n)	Theta(n log n)	O(1)	No	Yes	No
Distribution Counting	Theta(n + R)	Theta(n + R)	Theta(n + R)	O(n + R)	Yes	No	No (Requires bounded integer range R)

Index & Sorting Detail	Step-by-Step Trace Rules & Exam Questions
[6.1.1] Core Jargon	• Stable: Equal elements preserve their original relative order • In-place: O(1) extra data structure space • Adaptive: Faster on pre-sorted input
[6.3.1] Hoare's Partition (Quicksort Trace)	1. Pick leftmost element as pivot p = A[lo]. Set pointers L = lo + 1, R = hi. 2. Creep L right while A[L] < p. Creep R left while A[R] >= p. 3. If L < R (both stopped on an inversion), swap(A[L], A[R]) and repeat Step 2. 4. When pointers cross (L >= R), stop and swap pivot A[lo] with boundary pointer. Partition takes Theta(n) time.
[6.3.3] Quicksort Pivot Improvements	• Problem: Fixed leftmost/rightmost pivot on already-sorted (or reverse-sorted) input creates 0 / (n-1) splits -> Worst-case Theta(n^2). • Fix 1 (Randomized Pivot) & Fix 2 (Median-of-Three: median of first, middle, last): Eliminates predictable bad inputs; expected time O(n log n). • Exam Q: Does random pivot change worst-case time complexity? NO — unlucky random choices still give O(n^2) theoretical worst case.
[6.3.4] Quicksort + Insertion Sort Hybrid Cutoff	• Optimization: Stop Quicksort recursion when subarray size <= k (e.g. 10–15), then run one final pass of Insertion Sort over the whole array. • Exam Q: Improves real runtime (avoids tiny recursive call overhead + cache friendly), but DOES NOT change asymptotic Big-O complexity.
[6.4.3] Mergesort Variants	• Top-Down: Recursive split in half. • Bottom-Up: Iterative merge of sizes 1 -> 2 -> 4 -> 8... • Natural (Adaptive) Mergesort: Identifies existing monotonically increasing runs first; Best-case Theta(n) on sorted input (fewer comparisons).
[6.5.1] Heapsort Trace	Phase 1: Bottom-up build Max-Heap in Theta(n). Phase 2: Repeat n-1 times: swap root A[1] with last heap item A[k], k--, downheap(1) in O(n log n). • Instability Reason: Swapping root A[1] with A[k] jumps equal elements over each other, destroying relative order.

[7.0] Graphs, Traversals, Shortest Paths & Minimum Spanning Trees

[7.1.3] Representation	Space Complexity	Check Edge (u, v)	Iterate Neighbours(u)	Best Suited For
Adjacency Matrix	Theta(|V|^2)	Theta(1)	Theta(|V|)	Dense Graphs (|E| ≈ |V|^2), Floyd-Warshall
Adjacency List	Theta(|V| + |E|)	O(deg(u)) <= O(|V|)	Theta(deg(u))	Sparse Graphs (|E| ≈ |V|), BFS/DFS/Dijkstra/Prim
Implicit Graph [7.1.2]	O(states in memory)	O(1) via generator	O(b) branching factor	Huge state spaces (Milk pouring, Sudoku, Chess): time O(b^d)

Algorithm & Index	Queue / Structure	Time Complexity	Target Problem, Update Rule & Limitations
[7.2.1] DFS	Stack (LIFO) / Recursion	List: O(|V| + |E|) Matrix: O(|V|^2)	• Finds connected components (islands), cycles, topological sort (reverse post-order on DAG) • Does NOT guarantee shortest path. Max stack depth = |V| on a Line/Path Graph (A-B-C-D...)
[7.2.2] BFS	Queue (FIFO)	List: O(|V| + |E|) Matrix: O(|V|^2)	• Guarantees shortest path (minimum edges) on UNWEIGHTED graphs • Max queue length = |V|-1 on a Star Graph (source connected directly to all other nodes)
[7.3.1] UCS	Priority Queue	O((|E|+|V|) log |V|)	• Single-source to SINGLE-destination shortest path on weighted graphs (stops when target popped)
[7.3.2] Dijkstra	Priority Queue dist[], prev[]	Binary Heap: O((|E|+|V|) log |V|) Array PQ: O(|V|^2)	• Single-source to ALL-destinations shortest path (Non-negative edge weights ONLY: w(u,v) >= 0) • Relaxation: if dist[u] + w(u,v) < dist[v] -> dist[v] = dist[u] + w(u,v); prev[v] = u; • Path Reconstruction: Trace backwards from destination to source via prev[]
[7.3.4] Bellman-Ford	Edge List dist[], prev[]	O(|V| * |E|)	• Handles negative-weight edges (relaxes all |E| edges |V|-1 times) • FAILS if Negative-Weight Cycle exists (both Dijkstra & Bellman-Ford break on negative cycles)
[7.3.5] A* Search	Priority Queue f(v) = g(v) + h(v)	Depends on heuristic	• Priority = Cost_from_start g(v) + Heuristic_to_goal h(v) • Optimal ONLY IF h(v) is Admissible: 0 <= h(v) <= true_remaining_cost(v -> goal) (never overestimates)
[7.3.6] Floyd-Warshall	2D Matrices: D[V][V], Next[V][V]	Theta(|V|^3) always (Space: Theta(|V|^2))	• All-Pairs Shortest Path (APSP). Works on negative edges; independent of graph density • Loop order: for k (intermediate) -> for i (start) -> for j (end): if D[i][k] + D[k][j] < D[i][j]: D[i][j] = D[i][k] + D[k][j]; Next[i][j] = Next[i][k]; • Negative Cycle Detection: Check if any diagonal entry D[i][i] < 0 after completion
[7.4.2] Prim (MST)	Priority Queue dist[], prev[]	Binary Heap: O((|E|+|V|) log |V|)	• Vertex-centric Minimum Spanning Tree. Identical code to Dijkstra EXCEPT priority update: if w(u,v) < dist[v] -> dist[v] = w(u,v); prev[v] = u; (Single edge weight only, NO accumulation!) • Disconnected Graph Fix [7.4.3]: Re-run Prim on any remaining unvisited non-MST vertex until all visited
[7.4.4] Kruskal (MST)	Sorted Edges + Disjoint Set	O(|E| log |E|) (or O(|E| log |V|))	• Edge-centric MST: Sort edges by weight ascending; add edge (u,v) if find(u) ≠ find(v) until |V|-1 edges • Disjoint Set (Union-Find) Optimizations [7.4.5]: 1. Path Compression (point nodes directly to root during find) 2. Union-by-Rank (attach shorter tree under root of taller tree) -> near O(1) per edge

High-Frequency Graph Exam Questions	Exact Exam Answer / Proof
[7.3.3] Why NOT fix negative edges in Dijkstra by adding a large positive constant +C to every edge?	FALSE / INVALID: A path with k edges gets +k*C added to its total cost. Paths with more edges are penalized more than paths with fewer edges, changing the relative ranking of paths and returning wrong shortest paths.

High-Frequency Graph Exam Questions	Exact Exam Answer / Proof
[7.3.8] All-Pairs Shortest Path: Run |V| × Dijkstra vs Floyd-Warshall?	• |V| × Dijkstra takes O(|V|(|E|+|V|) log |V|). • Sparse Graph (|E| ≈ |V|, no negative edges): |V| × Dijkstra is O(|V|^2 log |V|), which is FASTER than Floyd-Warshall Theta(|V|^3). • Dense Graph (|E| ≈ |V|^2) or Negative Edges: |V| × Dijkstra is O(|V|^3 log |V|), so Floyd-Warshall Theta(|V|^3) is FASTER.
[7.4.1] Minimum Fatness Path (minimizing the max edge weight along a path between u and v)	1. Best Data Structure: The Minimum Spanning Tree (MST) guarantees containing the minimum fatness path between any pair of vertices. 2. Fastest Lookup on MST: Since an MST is sparse (|E| = |V| - 1), run Dijkstra / BFS / DFS on the MST in O(|V| log |V|) or O(|V|) rather than Floyd-Warshall Theta(|V|^3).

[8.0] Algorithm Design Paradigms & P / NP Complexity Theory

Index & Concept	Definition & Key Characteristics	Classic Course Examples
[8.1.1] Brute-Force	Exhaustively enumerates all candidate solutions	0-1 Knapsack (check all 2^n subsets -> O(2^n)), TSP all permutations O(n!)
[8.1.1] Divide & Conquer	Splits problem into independent subproblems, solves recursively, combines	Mergesort, Quicksort, Binary Search
[8.1.1] Greedy	Makes locally optimal choice at each step without backtracking	Dijkstra, Prim, Kruskal, Huffman Coding (Fails to guarantee optimal for 0-1 Knapsack)
[8.1.1] Dynamic Programming	Solves overlapping subproblems bottom-up using table (optimal substructure)	Floyd-Warshall Theta(|V|^3), 0-1 Knapsack O(n * Capacity) (pseudopolynomial)
[8.2.1] Deterministic vs Non-Deterministic	• Deterministic: Every state has at most 1 uniquely defined next step • Non-Deterministic: Theoretical machine that branches/guesses multiple next states simultaneously	All physical computers are deterministic; non-deterministic machines are theoretical models
[8.2.2] Decision Problem	A problem whose output is strictly YES (True) or NO (False)	Optimization problems are cast as decision problems (e.g. 'Is there a path of length <= k?')
[8.2.2] Class P vs Class NP	• Class P: Decision problems solvable deterministically in Polynomial Time O(n^k) • Class NP: Decision problems where a given candidate solution (certificate) can be VERIFIED deterministically in Polynomial Time (or solved non-deterministically in poly-time)	• P subset of NP • Verifying a completed Sudoku grid or checking if a given path visits all nodes with cost <= K takes polynomial time -> in NP
[8.2.3] NP-Hard & NP-Complete	• NP-Hard: Every problem in NP can be reduced to it in polynomial time • NP-Complete: Both (1) in NP and (2) NP-Hard. If ANY NP-Complete problem is solved in poly-time, then P = NP	Classic NP-Complete / NP-Hard problems: • Travelling Salesperson Problem (TSP) • Hamiltonian Circuit • 0-1 Knapsack • SAT • Longest Simple Path